

```
#<----- MODULE 1: DATASET LOADING ----->
# <----- IMPORTING LIBRARIES ----->

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# <----- IMPORTING FILE ----->

from google.colab import files

uploaded = files.upload()
```

```
# <----- READING DATASET ----->

file_name = list(uploaded.keys())[0]

df = pd.read_csv(file_name)

print("Dataset Loaded Successfully!")
```

```
# <----- DISPLAYING FIRST 5 RECORDS ----->

df.head()
```

```
# <----- DISPLAYING LAST 5 RECORDS ----->

df.tail()
```

```
#<----- MODULE 2: DATA EXPLORATION ----->
# <----- CHECKING DATASET SHAPE ----->

print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
```

```
# <----- DISPLAYING COLUMN NAMES ----->

print("Column Names:")
print(df.columns.tolist())
```

```
# <----- CHECKING DATA TYPES ----->

print("Data Types of Each Column:")
print(df.dtypes)
```

```
# <----- CHECKING MISSING VALUES ----->

print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
# <----- STATISTICAL SUMMARY ----->

df.describe()
```

```
# <----- COMPLETE DATASET INFORMATION ----->

df.info()
```

```
#<----- MODULE 3: DATA CLEANING ----->
# <----- CHECKING MISSING VALUES ----->

print("Missing Values in Each Column:")
print(df.isnull().sum())
```

```
# <----- CHECKING TOTAL MISSING VALUES ----->

total_missing = df.isnull().sum().sum()

print("Total Missing Values:", total_missing)
```

```
# <----- CHECKING DUPLICATE RECORDS ----->
```

```
duplicate_count = df.duplicated().sum()
```

```
print("Number of Duplicate Records:", duplicate_count)
```

```
# <----- DISPLAYING DUPLICATE RECORDS ----->
```

```
df[df.duplicated()]
```

```
# <----- CHECKING DUPLICATE STUDENT IDs ----->
```

```
duplicate_ids = df['Student_ID'].duplicated().sum()
```

```
print("Duplicate Student IDs:", duplicate_ids)
```

```
# <----- CHECKING INCORRECT AGE VALUES ----->
```

```
invalid_age = df[
    (df['Age'] < 0) |
    (df['Age'] > 100)
]
```

```
print("Incorrect Age Records:")
print(invalid_age)
```

```
# <----- CHECKING INCORRECT ATTENDANCE VALUES ----->
```

```
invalid_attendance = df[
    (df['Attendance (%)'] < 0) |
    (df['Attendance (%)'] > 100)
]
```

```
print("Incorrect Attendance Records:")
print(invalid_attendance)
```

```
# <----- CHECKING INCORRECT MIDTERM SCORES ----->
```

```
invalid_midterm = df[
    (df['Midterm_Score'] < 0) |
    (df['Midterm_Score'] > 100)
]
```

```
print("Incorrect Midterm Score Records:")
print(invalid_midterm)
```

```
# <----- CHECKING INCORRECT FINAL SCORES ----->
```

```
invalid_final = df[
    (df['Final_Score'] < 0) |
    (df['Final_Score'] > 100)
]
```

```
print("Incorrect Final Score Records:")
print(invalid_final)
```

```
# <----- REMOVING DUPLICATE RECORDS ----->
```

```
df = df.drop_duplicates()
```

```
print("Duplicate records removed successfully.")
```

```
# <----- VERIFYING DATA AFTER CLEANING ----->
```

```
print("Remaining Duplicate Records:", df.duplicated().sum())
print("Total Missing Values:", df.isnull().sum().sum())
```

```
#<----- MODULE 4: DATA PREPROCESSING ----->
```

```
# <----- PREPARING NUMERICAL DATA ----->
```

```
numeric_columns = df.select_dtypes(
    include=np.number
).columns
```

```
print("Numerical Columns:")
print(numeric_columns.tolist())
```

```
# <----- CHECKING NUMERICAL DATA ----->
```

```
print("Numerical Data Summary:")
print(df[numeric_columns].describe())
```

```
# <----- DETECTING OUTLIERS USING IQR ----->
```

```
for column in numeric_columns:
```

```
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR
```

```
    outliers = df[
        (df[column] < lower_limit) |
        (df[column] > upper_limit)
    ]
```

```
    print(column, "-> Outliers:", len(outliers))
```

```
# <----- DETECTING OUTLIERS USING IQR ----->
```

```
for column in numeric_columns:
```

```
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
```

```
    IQR = Q3 - Q1
```

```
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR
```

```
    outliers = df[
        (df[column] < lower_limit) |
        (df[column] > upper_limit)
    ]
```

```
    print(column, "-> Outliers:", len(outliers))
```

```
# <----- CREATING AVERAGE FOR PERFORMANCE ANALYSIS ----->
```

```
df['Average'] = df[
    ['Midterm_Score', 'Final_Score']
].mean(axis=1)
```

```
print("Average column created successfully.")
```

```
df[
    ['Student_ID', 'Midterm_Score', 'Final_Score', 'Average']
].head()
```

```
# <----- VERIFYING PREPROCESSED DATA ----->
```

```
print("Dataset Shape:", df.shape)
print("Total Missing Values:", df.isnull().sum().sum())
print("Total Duplicate Records:", df.duplicated().sum())
```

```
#<----- MODULE 5: PERFORMANCE ANALYSIS ----->
```

```
# <----- CALCULATING STUDENT AVERAGE ----->
```

```
df['Average'] = df[
    ['Midterm_Score', 'Final_Score']
].mean(axis=1)
```

```
print("Average calculated successfully!")
```

```
df[
    ['Student_ID', 'First_Name', 'Last_Name',
     'Midterm_Score', 'Final_Score', 'Average']
].head()
```

```
# <----- CHECKING EXISTING GRADES ----->
```

```
print("Grade Categories:")
print(df['Grade'].unique())

print("\nGrade Distribution:")
print(df['Grade'].value_counts())
```

```
# <----- GENERATING CALCULATED GRADE ----->

def calculate_grade(average):
    if average >= 90:
        return 'A+'
    elif average >= 80:
        return 'A'
    elif average >= 70:
        return 'B'
    elif average >= 60:
        return 'C'
    elif average >= 50:
        return 'D'
    else:
        return 'F'

df['Calculated_Grade'] = df['Average'].apply(calculate_grade)

print("Calculated Grade generated successfully!")

df[
    ['Student_ID', 'Average', 'Grade', 'Calculated_Grade']
].head()
```

```
# <----- SORTING STUDENTS BY PERFORMANCE ----->

df_sorted = df.sort_values(
    by='Average',
    ascending=False
)

print("Students Sorted by Average Score:")

df_sorted[
    ['Student_ID', 'First_Name', 'Last_Name',
     'Average', 'Grade']
].head(10)
```

```
# <----- FILTERING HIGH PERFORMING STUDENTS ----->

high_performers = df[
    df['Average'] >= 75
]

print("High Performing Students:")

high_performers[
    ['Student_ID', 'First_Name', 'Last_Name', 'Average', 'Grade']
]
```

```
# <----- IDENTIFYING TOPPER ----->

topper_index = df['Average'].idxmax()

topper = df.loc[topper_index]

print("Topper Information:")
print("Student ID:", topper['Student_ID'])
print("Name:", topper['First_Name'], topper['Last_Name'])
print("Average:", topper['Average'])
print("Grade:", topper['Grade'])
```

```
# <----- DEPARTMENT-WISE PERFORMANCE ANALYSIS ----->

department_performance = df.groupby(
    'Department'
)['Average'].mean().sort_values(
    ascending=False
)

print("Department-wise Average Performance:")
print(department_performance)
```

```
# <----- MIDTERM AND FINAL PERFORMANCE COMPARISON ----->

print("Average Midterm Score:",
      df['Midterm_Score'].mean())

print("Average Final Score:",
      df['Final_Score'].mean())

if df['Final_Score'].mean() > df['Midterm_Score'].mean():
    print("Overall final performance is higher than midterm performance.")
elif df['Final_Score'].mean() < df['Midterm_Score'].mean():
    print("Overall midterm performance is higher than final performance.")
else:
    print("Midterm and final performance are equal.")
```

```
# <----- FINAL PERFORMANCE ANALYSIS ----->

print("Total Students:", len(df))
print("Average Student Score:", round(df['Average'].mean(), 2))
print("Highest Average Score:", round(df['Average'].max(), 2))
print("Lowest Average Score:", round(df['Average'].min(), 2))

print("\nPerformance Analysis Completed Successfully!")
```

```
#<----- MODULE 6: VISUALIZATION ----->
# <----- GRADE DISTRIBUTION BAR GRAPH ----->

grade_counts = df['Grade'].value_counts()

plt.figure(figsize=(8, 5))

grade_counts.plot(kind='bar')

plt.title("Grade Distribution")
plt.xlabel("Grade")
plt.ylabel("Number of Students")
plt.xticks(rotation=0)

plt.show()
```

```
# <----- GRADE DISTRIBUTION PIE CHART ----->

grade_counts = df['Grade'].value_counts()

plt.figure(figsize=(7, 7))

plt.pie(
    grade_counts,
    labels=grade_counts.index,
    autopct='%1.1f%%'
)

plt.title("Grade Distribution of Students")

plt.show()
```

```
# <----- MIDTERM VS FINAL PERFORMANCE COMPARISON ----->

average_scores = [
    df['Midterm_Score'].mean(),
    df['Final_Score'].mean()
]

subjects = ['Midterm Score', 'Final Score']

plt.figure(figsize=(8, 5))

plt.bar(subjects, average_scores)

plt.title("Midterm vs Final Performance")
plt.xlabel("Assessment")
plt.ylabel("Average Score")

plt.show()
```

```
# <----- DEPARTMENT-WISE PERFORMANCE COMPARISON ----->

department_average = df.groupby(
```

```
'Department'
)['Average'].mean()

plt.figure(figsize=(10, 5))

department_average.plot(kind='bar')

plt.title("Department-wise Average Performance")
plt.xlabel("Department")
plt.ylabel("Average Score")
plt.xticks(rotation=45)

plt.show()
```

```
# <----- STUDENT PERFORMANCE COMPARISON ----->

top_students = df.sort_values(
    by='Average',
    ascending=False
).head(10)

plt.figure(figsize=(10, 5))

plt.bar(
    top_students['Student_ID'].astype(str),
    top_students['Average']
)

plt.title("Top 10 Student Performance")
plt.xlabel("Student ID")
plt.ylabel("Average Score")
plt.xticks(rotation=45)

plt.show()
```

```
#<----- MODULE 7: REPORT GENERATION ----->
# <----- DATASET SUMMARY ----->

print("===== DATASET SUMMARY =====")

print("Total Students:", len(df))
print("Total Columns:", df.shape[1])
print("Total Missing Values:", df.isnull().sum().sum())
print("Total Duplicate Records:", df.duplicated().sum())
```

```
# <----- CLEANED DATASET ----->

print("===== CLEANED DATASET =====")

df.head(10)
```

```
# <----- STUDENT STATISTICS ----->

print("===== STUDENT STATISTICS =====")

print("Average Midterm Score:",
      round(df['Midterm_Score'].mean(), 2))

print("Average Final Score:",
      round(df['Final_Score'].mean(), 2))

print("Overall Average:",
      round(df['Average'].mean(), 2))

print("Highest Average:",
      round(df['Average'].max(), 2))

print("Lowest Average:",
      round(df['Average'].min(), 2))
```

```
# <----- TOPPER INFORMATION ----->

print("===== TOPPER INFORMATION =====")

topper_index = df['Average'].idxmax()
topper = df.loc[topper_index]

print("Student ID:", topper['Student_ID'])
print("Name:", topper['First_Name'], topper['Last_Name'])
```

```
print("Department:", topper['Department'])
print("Average:", round(topper['Average'], 2))
print("Grade:", topper['Grade'])
```

```
# <----- GRADE DISTRIBUTION ----->

print("===== GRADE DISTRIBUTION =====")

grade_distribution = df['Grade'].value_counts()

print(grade_distribution)
```

```
# <----- DEPARTMENT-WISE ANALYSIS ----->

print("===== DEPARTMENT-WISE ANALYSIS =====")

department_analysis = df.groupby(
    'Department'
)[['Midterm_Score', 'Final_Score', 'Average']].mean()

print(department_analysis.round(2))
```

```
# <----- PERFORMANCE INSIGHTS ----->

print("===== PERFORMANCE INSIGHTS =====")

overall_average = df['Average'].mean()
highest_average = df['Average'].max()
lowest_average = df['Average'].min()

print("Overall Average:", round(overall_average, 2))
print("Highest Average:", round(highest_average, 2))
print("Lowest Average:", round(lowest_average, 2))

print("Number of High Performers:",
      len(df[df['Average'] >= 75]))

print("Most Common Grade:",
      df['Grade'].mode()[0])
```

```
# <----- SAVING CLEANED DATASET ----->

df.to_csv(
    'Student_Performance_Cleaned.csv',
    index=False
)

print("Cleaned dataset saved successfully!")
```

```
# <----- GENERATING FINAL REPORT ----->

report = f"""
STUDENT PERFORMANCE DATA ANALYSIS REPORT
=====

DATASET SUMMARY
-----
Total Students: {len(df)}
Total Columns: {df.shape[1]}
Missing Values: {df.isnull().sum().sum()}
Duplicate Records: {df.duplicated().sum()}

STUDENT STATISTICS
-----
Average Midterm Score: {df['Midterm_Score'].mean():.2f}
Average Final Score: {df['Final_Score'].mean():.2f}
Overall Average: {df['Average'].mean():.2f}
Highest Average: {df['Average'].max():.2f}
Lowest Average: {df['Average'].min():.2f}

TOPPER INFORMATION
-----
Student ID: {topper['Student_ID']}
Name: {topper['First_Name']} {topper['Last_Name']}
Department: {topper['Department']}
Average: {topper['Average']:.2f}
Grade: {topper['Grade']}

GRADE DISTRIBUTION
```

```
-----  
{grade_distribution.to_string()}  
  
PERFORMANCE INSIGHTS  
-----  
High Performers: {len(df[df['Average'] >= 75])}  
Most Common Grade: {df['Grade'].mode()[0]}  
"""  
  
print(report)
```

```
# <----- SAVING FINAL REPORT ----->  
  
with open(  
    'Student_Performance_Final_Report.txt',  
    'w'  
) as file:  
    file.write(report)  
  
print("Final report generated successfully!")
```

```
# <----- DOWNLOADING FINAL FILES ----->  
  
from google.colab import files  
  
files.download('Student_Performance_Cleaned.csv')
```

```
# <----- DOWNLOADING FINAL REPORT ----->  
  
files.download('Student_Performance_Final_Report.txt')
```